INF220 / UNIDAD 3 / PRÁCTICA 03

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS · FACULTAD DE CIENCIAS PURAS · INGENIERÍA INFORMÁTICA

Práctica 3

Herencia en Java

Figuras geométricas planas

MATERIA	INF 220 – Programación Orientada a Objetos y Estructuras de Datos
UNIDAD	U3 – Herencia y Polimorfismo
MODALIDAD	Trabajo independiente (individual)
LENGUAJE / ENTORNO	Java (JDK Temurin 25) · Visual Studio Code + Extension Pack for Java
PUNTAJE	100 puntos
ESTUDIANTE	
FECHA DE ENTREGA	

01 OBJETIVOS

- Construir una jerarquía de clases en Java usando la palabra reservada `extends`.
- Reutilizar atributos y métodos de una clase padre en sus clases hijas, incluyendo herencia multinivel.
- Invocar constructores de la clase padre mediante `super(...)` y sobrescribir métodos con `@Override`.
- Integrar la jerarquía en un programa de consola con menú, entrada por teclado y una colección de objetos.

02 ENUNCIADO

Desarrolle un programa de consola en Java que permita **registrar figuras geométricas planas** y calcular su **área** y su **perímetro**. Las figuras deben organizarse en una jerarquía de clases usando **herencia**, de modo que los datos y el comportamiento comunes se escriban una sola vez en la clase padre y se reutilicen en las clases hijas.

El programa debe ofrecer un menú que permita registrar círculos, triángulos, rectángulos y cuadrados, y listar todas las figuras registradas mostrando sus datos y medidas calculadas.

DEFINICIÓN · HERENCIA

La herencia permite que una clase (hija o subclase) adquiera los atributos y métodos de otra clase (padre o superclase). En Java se declara con `extends`; la clase hija puede agregar nuevos miembros y **sobrescribir** los métodos heredados para adaptarlos a su caso.

03 JERARQUÍA DE CLASES

La jerarquía que debe implementar es la siguiente. Observe que `Cuadrado` no hereda directamente de `Figura`, sino de `Rectangulo` (herencia multinivel).

jerarquia.txt

```
Figura
├── Circulo
├── Triangulo
└── Rectangulo
    └── Cuadrado
```

04 ESPECIFICACIÓN DE LAS CLASES

Clase	Hereda de	Atributos	Métodos / comportamiento
Figura	—	`nombre`, `color` (protected)	Constructor, getters/setters, `area()` y `perimetro()` que devuelven 0, `toString()` con los datos comunes.
Circulo	Figura	`radio`	Sobrescribe `area()`, `perimetro()` y `toString()`.
Triangulo	Figura	`ladoA`, `ladoB`, `ladoC`	Sobrescribe `area()` (fórmula de Herón), `perimetro()` y `toString()`.
Rectangulo	Figura	`base`, `altura`	Sobrescribe `area()`, `perimetro()` y `toString()`.
Cuadrado	Rectangulo	— (usa los heredados)	Constructor que recibe un `lado` y llama a `super(lado, lado)`. Solo sobrescribe `toString()`.

IMPORTANTE

`Cuadrado` **no debe** redefinir `area()` ni `perimetro()`: debe reutilizar los que hereda de `Rectangulo`. El objetivo es comprobar que la herencia evita duplicar código.

▸ **Fórmulas de referencia**

Figura	Área	Perímetro
Círculo	$\pi \cdot r^2$	$2 \cdot \pi \cdot r$
Triángulo	$\sqrt{s(s-a)(s-b)(s-c)}$, con $s = (a + b + c) / 2$	$a + b + c$
Rectángulo	base · altura	2 · (base + altura)
Cuadrado	lado^2 (heredada de Rectángulo)	4 · lado (heredada de Rectángulo)

05 REQUISITOS TÉCNICOS

R1. Uso de `extends` en todas las clases hijas y de `super(...)` en sus constructores.

R2. Atributos `nombre` y `color` declarados como `protected` en `Figura`; los atributos propios de cada hija, como `private` con sus getters y setters.

R3. Sobrescritura de `area()`, `perimetro()` y `toString()` con la anotación `@Override`.

R4. El `toString()` de cada clase hija debe llamar a `super.toString()` y agregar sus propios datos, área y perímetro.

R5. Validar que todas las medidas ingresadas sean mayores que cero; si no lo son, mostrar un mensaje y no registrar la figura.

R6. Mostrar los resultados con dos decimales (por ejemplo, con `String.format("%.2f", valor)`) y usar `Math.PI` y `Math.sqrt`.

R7. Cada clase debe estar en su propio archivo `.java`.

> **NOTA**
>
> Si los tres lados ingresados no pueden formar un triángulo (por ejemplo 1, 2 y 10), la fórmula de Herón produce un resultado no válido (`NaN`). Verificar la desigualdad triangular (a + b > c, a + c > b, b + c > a) no es obligatorio, pero es una buena práctica.

06 PROGRAMA PRINCIPAL

Cree la clase `App` con el método `main`. Las figuras se almacenan en un `ArrayList<Figura>` y el programa muestra el siguiente menú de forma repetida usando `Scanner`, hasta que el usuario elija salir:

menú

```
===== FIGURAS GEOMÉTRICAS =====
1. Registrar un círculo
2. Registrar un triángulo
3. Registrar un rectángulo
4. Registrar un cuadrado
5. Listar figuras registradas
6. Salir
Elija una opción:
```

Para cada registro se solicitan el nombre, el color y las medidas correspondientes. La opción 5 recorre la lista e imprime el `toString()` de cada figura. Un ejemplo orientativo de la salida esperada:

salida de ejemplo

```
Elija una opción: 4
Nombre: Baldosa
Color: Gris
Lado: 3
Cuadrado registrado correctamente.

Elija una opción: 5
--- Figuras registradas (2) ---
Nombre: Rueda | Color: Negro | Radio: 2.00 | Área: 12.57 | Perímetro: 12.57
Nombre: Baldosa | Color: Gris | Base: 3.00 | Altura: 3.00 | Lado: 3.00 | Área: 9.00 |
Perímetro: 12.00
```

07 ENTREGABLES

- Proyecto completo en Visual Studio Code, comprimido como `P3_ApellidoNombre.zip`.
- Capturas de pantalla de la ejecución que muestren el registro de cada tipo de figura, al menos un caso de medida inválida y el listado final.

08 CRITERIOS DE EVALUACIÓN

Criterio	Descripción	Pts
Jerarquía de clases	Las cinco clases existen, usan `extends` correctamente y respetan la jerarquía (incluida la herencia multinivel).	30
Constructores	Todos los constructores de las hijas invocan a `super(...)`; `Cuadrado` llama a `super(lado, lado)`.	20
Sobrescritura de métodos	`area()`, `perimetro()` y `toString()` se sobrescriben con `@Override`; `toString()` reutiliza `super.toString()`; `Cuadrado` no duplica cálculos.	20

Criterio	Descripción	Pts
Cálculos y validaciones	Fórmulas correctas, resultados con dos decimales y rechazo de medidas no positivas.	15
Menú y entregables	Menú funcional con `ArrayList<Figura>`, capturas completas y respuestas a las preguntas.	15
Total		**100**

09 BIBLIOGRAFÍA

- Deitel, P. y Deitel, H. (2017). Java: cómo programar (10.ª ed.). Pearson. Capítulo 9: Programación orientada a objetos: herencia.
- Schildt, H. (2022). Java: The Complete Reference (12.ª ed.). McGraw-Hill. Capítulo 8: Inheritance.
- Oracle. (s. f.). The Java Tutorials: Inheritance. https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html